

SPC: self-play-критик для оценки рассуждений. Обзор статьи и эксперимент

Статья: Chen et al., *SPC: Evolving Self-Play Critic via Adversarial Games for LLM Reasoning*, NeurIPS 2025, [arXiv:2504.19162](https://arxiv.org/abs/2504.19162).

Оригинальная статья

Авторы ставят перед собой цель — научить модель находить ошибки в пошаговых математических решениях без ручной разметки каждого шага. Они берут две копии Qwen2.5-7B-Instruct, дообучают их на размеченных данных (стартовое дообучение, SFT) и сталкивают в игре двух ролей.

- **Генератор-обманщик** переписывает один шаг корректного решения в неверный, так чтобы тот не выдавал себя, и пытается обмануть критика.
- **Критик**, видя задачу, предыдущие шаги и проверяемый шаг, после короткого разбора отвечает `Correct` или `Incorrect`.

За раунд разыгрываются десятки тысяч партий. Награду определяет исход — очко достаётся тому, кто переиграл, — и по ней обе модели дообучаются через PPO: генератор учится прятать ошибки тоньше, критик находить их точнее. После SFT разметка больше не нужна, работает self-play.

Авторы выложили две версии критика — Critic-0 (после SFT) и Critic-2 (после двух раундов); генератор не опубликован.

Задачи

Центральная задача — построить надёжного пошагового критика без ручной разметки шагов. Пошаговая оценка нужна, чтобы награждать модель за каждый шаг, а не только за финальный ответ. Сильные критики такого рода (process-reward-модели, PRM) обучаются на тысячах размеченных людьми шагов (PRM800K от OpenAI), и эта разметка не масштабируется. Отсюда две цели: порождать обучающий сигнал автоматически и не дать ему упрощаться по мере роста критика. Целью был критик, который через self-play догоняет на ProcessBench PRM.

Сильные стороны

Разметка нужна только для SFT — дальше процесс обходится без человека. Награда за каждый шаг плотнее награды за финальный ответ, поэтому обучение сходится быстрее. Архитектура простая: одна 7B-модель в обеих ролях, без отдельной reward model и value-функции. При этом результат подтвердил эффективность — на ProcessBench Critic-2 даёт около 78% F1, на уровне PRM, обученных на размеченных данных.

Слабые стороны

Критик ловит лишь те ошибки, которые умеет вставлять текущий генератор, — это искусственное распределение, тогда как ProcessBench собран из настоящих ошибок моделей. Расхождение train/test заложено в самом методе, но в статье не обсуждается.

В релизе две версии критика из трёх, а генератор закрыт — нельзя ни воспроизвести его сторону обучения, ни заглянуть за второй раунд.

Вся оценка сведена к одному числу — ProcessBench F1, по которому не видно, что изменила самоигра: вырос recall или просто упали ложные срабатывания на корректных шагах. Это я и проверил.

Гипотеза эксперимента

В каждом обучающем примере критик видит шаг с подложенной ошибкой, так что после многих раундов его prior должен смещаться к ответу «Incorrect». Предсказание: Critic-2 точнее ловит настоящие ошибки (выше recall), но чаще принимает корректные шаги за неверные (ниже accuracy). Поэтому я раскладываю общую метрику на recall на ошибочных шагах и accuracy на корректных.

Эксперимент

Беру обе открытые версии (judge/SPC-Critic-0, judge/SPC-Critic-2) и первые 40 объектов каждого подмножества ProcessBench (gsm8k, math, olympiadbench, omnimath). Каждый разворачиваю в пошаговые пробы по схеме SPC — 296 проб на критика: 136 корректных шагов, 160 с ошибкой. Оба критика идут по одному набору, поэтому сравнение парное. Промпт и формат ответа — из репозитория авторов,

декодирование жадное ($T=0$), `max_new_tokens=512`, инференс через vLLM. Отступления от оригинала: ProcessBench пересобран из сырого HF-набора, выборка $N=296$ против ~ 3400 .

метрика	Critic-0 (SFT)	Critic-2 (RL)	Δ
recall на ошибочных шагах	0.731	0.711	-0.020
специфичность на корректных шагах	0.674	0.859	+0.185
ProcessBench F1 (гарм. среднее, headline)	0.701	0.778	+0.077
арифм. среднее (balanced accuracy)	0.705	0.781	+0.076

ProcessBench F1 у Critic-2 — 0.778 против 0.777 в статье: расхождение меньше 0.1 пункта на выборке в 12 раз меньшей. Пайплайн воспроизводит авторские цифры.

Результаты

По гипотезе recall должен был вырасти, а специфичность — упасть. Вышло наоборот: recall чуть снизился (на 2 пункта), а специфичность (точность на корректных шагах) выросла на 18.5. Ниже сравнение предсказаний C0 и C2:

истинный класс шага	$C0 < C2$	$C0 > C2$	Σ
шаги без ошибки ($n=136$)	31	6	+25
шаги с ошибкой ($n=160$)	22	26	-4

C0 — Critic-0, C2 — Critic-2; « $C0 < C2$ » — C0 ошибся, а C2 ответил верно; « $C0 > C2$ » — наоборот; Σ = (стало лучше) — (стало хуже).

На корректных шагах второй раунд даёт сильный прирост качества (+25), на ошибочных — немного ломает (-4). Значит, прирост даёт не лучшая детекция ошибок, а то, что Critic-2 перестал ошибочно отвергать корректные шаги: критик стал *снисходительнее*, и гипотеза не подтвердилась.

Ограничения

- 296 проб против ~ 3400 в полном ProcessBench; доверительных интервалов нет, поэтому небольшие сдвиги (особенно recall, -2 пункта) могут лежать в пределах шума.
- ProcessBench развёрнут в пошаговые пробы моей схемой, а не оригинальными eval-данными авторов; абсолютные числа могут смещаться — надёжно лишь сравнение Critic-0 vs Critic-2.
- В релизе лишь Critic-0 и Critic-2, промежуточные раунды недоступны, поэтому вывод о направлении тренда опирается всего на две точки.
- Жадное декодирование без повторов и единственный домен (математика ProcessBench); разброс по сэмплингованию и перенос на другие задачи не проверялись.

Следующие шаги

Эксперимент воспроизвёл число из статьи, но вскрыл его природу: прирост даёт рост специфичности (критик стал мягче), а не более чуткая детекция ошибок. Пока это в плюс, но направление опасное — с ростом числа итераций критик может начать терять настоящие ошибки быстрее, чем выигрывает на корректных. Отсюда главный вопрос: монотонен ли этот тренд, есть ли раунд, где самоигра переусердствует, и почему критик смягчается.

Ответить на него можно следующими экспериментами:

- Прогнать промежуточные раунды (Critic-1, Critic-3, ...) — их придётся дообучить, поскольку в релизе только Critic-0 и Critic-2: подтвердится ли тренд и есть ли точка, где recall начинает падать.
- Разложить пропущенные ошибки по типам (описка, неверная формула, путаница со знаком, пропущенный шаг) — это покажет, куда сместилось распределение генератора.

Источники

- [1] J. Chen и др. *SPC: Evolving Self-Play Critic via Adversarial Games for LLM Reasoning*. NeurIPS 2025. arXiv:2504.19162.
- [2] Qwen Team. *Qwen2.5 Technical Report*. 2024. arXiv:2412.15115.
- [3] J. Schulman и др. *Proximal Policy Optimization Algorithms*. 2017. arXiv:1707.06347.
- [4] H. Lightman и др. *Let's Verify Step by Step*. 2023. arXiv:2305.20050. (датасет PRM800K)
- [5] C. Zheng и др. *ProcessBench: Identifying Process Errors in Mathematical Reasoning*. 2024. arXiv:2412.06559.
- [6] W. Kwon и др. *Efficient Memory Management for LLM Serving with PagedAttention (vLLM)*. SOSP 2023. arXiv:2309.06180.